Beacon — Knowledge & Decision-Support Layer

Ask internal business information in plain language — grounded, traceable, decision-aware

internal AI knowledge layer

The problem

A consulting firm's working knowledge is scattered across project records, methodology playbooks, per-client context and past deliverables. Teams lose time searching those sources and coordinating information by hand. The firm wanted an internal AI layer that helps people **access, understand and use** that information efficiently — and supports operational decisions — without becoming a black box. An internal business platform, not a public chatbot.

The build

Beacon is a FastAPI service with one query pipeline that handles two shapes of question. A plain question returns a grounded answer; an operational decision question returns a structured **decision brief**. Either way, retrieval is federated across four knowledge sources and every answer carries the exact document ids it rests on — full traceability by default. Synthesis runs on Claude when an API key is present and a deterministic extractive synthesizer otherwise; conversation memory resolves follow-up questions.

Architecture

```
Consultant — plain question or operational decision
   └─▶ Beacon API · FastAPI · /api/query
         └─▶ Federated retrieval · hybrid TF-IDF + keyword
               └─▶ 4 sources: project records · methodology · client KB · deliverables
         └─▶ Answer synthesis · Claude | deterministic extractive fallback
   decision question? ─▶ decision brief (recommendation + grounding + review gate)
                      ─▶ otherwise: grounded answer  ·  every source cited
```

Retrieval, synthesis, memory and decision logic are separate modules — the seams a production deployment uses to plug in live connectors and a real vector store, with the rest of the system unchanged.

Why it matters: nothing is asserted without a source — every response lists the documents it used; decision questions with thin or partial evidence raise a human-review flag instead of asserting false confidence; and the extractive fallback means a provider outage degrades answer quality rather than breaking the platform.

Verification

A 20-test suite covers retrieval correctness, the synthesis fallback, decision detection, conversation memory and the HTTP API end to end. The live demo was driven and verified in-browser.

Stack

Python · FastAPI · Claude · hybrid retrieval / RAG · pytest · Vercel

Waseem Iftikhar · AI / Backend Engineer

Portfolio case study